

Olist Brazilian E-Commerce

Data Cleaning Log — Phase 0 & 1

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle)

9 CSV files | 6 Tasks | Run top to bottom in Google Colab

T1 Load All 9 CSV Files

WHAT WE DID

Import pandas and read all 9 Olist CSV files from Google Drive into separate DataFrames — customers, orders, order_items, payments, reviews, products, sellers, geolocation, and category translation.

WHY WE DID IT

The dataset is split across 9 files by design, like database tables. We need all of them in memory at the same time before we can clean or join them.

IMPACT ON DATA

No data is changed yet. Just creates 9 named DataFrames. Sets the foundation for every task that follows — nothing else can run without this step.

Code

```
from google.colab import drive
import pandas as pd
drive.mount('/content/drive')

# Change only this line if your files are in a different folder
FOLDER = '/content/drive/MyDrive/Datasets/'
```

```

customers = pd.read_csv(FOLDER + 'olist_customers_dataset.csv')
orders = pd.read_csv(FOLDER + 'olist_orders_dataset.csv')
order_items = pd.read_csv(FOLDER + 'olist_order_items_dataset.csv')
order_payment = pd.read_csv(FOLDER + 'olist_order_payments_dataset.csv')
order_review = pd.read_csv(FOLDER + 'olist_order_reviews_dataset.csv')
products = pd.read_csv(FOLDER + 'olist_products_dataset.csv')
sellers = pd.read_csv(FOLDER + 'olist_sellers_dataset.csv')
geolocation = pd.read_csv(FOLDER + 'olist_geolocation_dataset.csv')
product_category = pd.read_csv(FOLDER + 'product_category_name_translation.csv')

```

Extra cleaning on the customers table

```

# Remove duplicate rows
customers = customers.drop_duplicates()

# Lowercase city names – 'Sao Paulo' and 'sao paulo' should be the same city
customers['customer_city'] = customers['customer_city'].str.lower().str.strip()

# ZIP code was stored as a number – convert to 5-digit string
# Example: 1151 becomes "01151" (prevents losing the leading zero)
customers['customer_zip_code_prefix'] = (
    customers['customer_zip_code_prefix'].astype(str).str.zfill(5)
)

```

T2 Validate Relationships Between Tables

WHAT WE DID

Check that every ID in a child table (like `order_id` in `order_items`) actually exists in the parent table (`orders`). Records that do not match are called orphans.

WHY WE DID IT

If orphan records exist and we merge tables with them, we either silently lose rows or get rows filled with empty `NaN` values — corrupting the entire analysis with no error message at all.

IMPACT ON DATA

No rows are deleted or changed. This is a quality audit only. All 6 checks return 0 for the clean Olist dataset, confirming that all merges in Task 5 are fully safe.

Code

```

checks = {

```

```

'order_items -> orders (order_id)' :
(~order_items['order_id'].isin(orders['order_id'])).sum(),

'order_payment -> orders (order_id)' :
(~order_payment['order_id'].isin(orders['order_id'])).sum(),

'order_review -> orders (order_id)' :
(~order_review['order_id'].isin(orders['order_id'])).sum(),

'orders -> customers (customer_id)':
(~orders['customer_id'].isin(customers['customer_id'])).sum(),

'order_items -> sellers (seller_id)' :
(~order_items['seller_id'].isin(sellers['seller_id'])).sum(),

'order_items -> products (product_id)':
(~order_items['product_id'].isin(products['product_id'])).sum(),
}

for check, count in checks.items():
status = "OK" if count == 0 else "WARNING – orphans found!"
print(f'{status} | {check} -> orphans: {count}')
```

T3 Parse Datetime Columns (3 files, 8 columns)

WHAT WE DID

Convert 8 timestamp columns stored as plain text strings like "2017-09-13 08:59:02" into proper pandas datetime objects using `pd.to_datetime()`. Done across orders (5 columns), order_review (2), and order_items (1).

WHY WE DID IT

Python treats dates stored as strings just like any other text — it has no idea they represent time. You cannot subtract strings to get a duration or extract month/year from them. Converting to datetime unlocks all time-based math.

IMPACT ON DATA

Column dtype changes from object (string) to datetime64. This is required for Task 6 — without this conversion, the date subtraction in Task 6 will throw a `TypeError` and the delivery features cannot be built.

Code

```

# Orders – 5 datetime columns
datetime_cols_orders = [
    'order_purchase_timestamp',
    'order_approved_at',
    'order_delivered_carrier_date',
    'order_delivered_customer_date',
```

```

'order_estimated_delivery_date'
]
for col in datetime_cols_orders:
orders[col] = pd.to_datetime(orders[col])

# Order reviews – 2 datetime columns
order_review['review_creation_date'] =
pd.to_datetime(order_review['review_creation_date'])

order_review['review_answer_timestamp'] =
pd.to_datetime(order_review['review_answer_timestamp'])

# Order items – 1 datetime column
order_items['shipping_limit_date'] = pd.to_datetime(order_items['shipping_limit_date'])

```

T4 Aggregate Payments Per Order (Handle Split Payments)

WHAT WE DID

The payments table has one row per payment method per order. Some orders are paid with multiple methods (e.g. credit card + voucher), giving them 2 or more rows. We collapse these into one row per order using groupby and agg.

WHY WE DID IT

If we merge the raw payments table directly with orders, an order paid in 2 methods becomes 2 rows in the master table — doubling its revenue and count. This silently corrupts the entire analysis with no warning or error.

IMPACT ON DATA

Reduces the payment table from ~103,000 rows to ~99,000 rows (one per order). Creates 3 new enriched columns: total_payment_value, payment_installments, and payment_types_used.

Code

```

payments_agg = order_payment.groupby('order_id').agg(
total_payment_value = ('payment_value', 'sum'),
payment_installments = ('payment_installments', 'max'),
# Joins all payment types into one string e.g. "credit_card,voucher"
payment_types_used = ('payment_type', lambda x: ','.join(sorted(x.unique()))
).reset_index()

print(f'Before: {len(order_payment)} rows -> After: {len(payments_agg)} rows')

```

T5 Build Master Joined Table — All Tables Into One

WHAT WE DID

Merge all tables together into one wide master DataFrame using a chain of left joins. The orders table is the base. We attach customers, payments, item summaries, and review scores onto it one by one.

WHY WE DID IT

Analysis questions span multiple tables. Instead of querying 9 separate DataFrames every time, one master table lets you answer any question in a single line. This becomes the single source of truth for all Phase 2 EDA.

IMPACT ON DATA

Creates a new master DataFrame with ~99,000 rows and 20+ columns. Individual tables remain unchanged. Every merge uses `how="left"` so no orders are ever lost — even if they have no review or payment record.

Code

```
# Step 1: orders + customers (join key: customer_id)
master = orders.merge(customers, on='customer_id', how='left')

# Step 2: + aggregated payments (join key: order_id)
master = master.merge(payments_agg, on='order_id', how='left')

# Step 3: + order items summary (aggregate first, then join on order_id)
items_agg = order_items.groupby('order_id').agg(
    total_items = ('order_item_id', 'count'),
    total_freight = ('freight_value', 'sum'),
    total_price = ('price', 'sum')
).reset_index()
master = master.merge(items_agg, on='order_id', how='left')

# Step 4: + review score only (join key: order_id)
master = master.merge(
    order_review[['order_id', 'review_score']],
    on='order_id', how='left'
)

print(f'Master table shape: {master.shape}')
```

T6 Engineer delivery_days, delay_days, is_late

WHAT WE DID

Create 3 new calculated columns using date subtraction: `delivery_days` (days from purchase to actual delivery), `delay_days` (actual vs estimated — negative means arrived early), and `is_late` (True/False flag for late orders).

WHY WE DID IT

The raw data only has raw timestamps — it does not tell you if delivery was fast or slow, early or late. These new columns translate raw dates into business-meaningful metrics that can be directly used in charts and analysis.

IMPACT ON DATA

Adds 3 new columns to the master table. `delivery_days` and `delay_days` are integers (negative = early). `is_late` is boolean. Cancelled orders have no delivery date so their values will be NaN — that is expected and correct.

Code

```
# Days from purchase to actual delivery
master['delivery_days'] = (
    master['order_delivered_customer_date'] - master['order_purchase_timestamp']
).dt.days

# Days late vs estimated (negative = delivered early)
master['delay_days'] = (
    master['order_delivered_customer_date'] - master['order_estimated_delivery_date']
).dt.days

# Boolean flag – True if order arrived after the promised date
master['is_late'] = master['delay_days'] > 0

# Quick delivery summary
print(f'Late delivery rate : {master["is_late"].mean():.1%}')
print(f'Avg delivery time : {master["delivery_days"].mean():.1f} days')
```

Cleaning Log — Summary of All 6 Tasks

Task	Name	What was done	Data impact
T1	Load CSVs	All 9 CSV files loaded from Google Drive	No change — foundational step

T1 b	Clean customers	Lowercase cities, fix ZIP dtype, drop duplicates	ZIP fixed, city names standardized
T2	Validate FKs	6 foreign key relationships checked for orphans	Audit only — 0 orphans found
T3	Parse datetimes	8 timestamp columns converted in 3 files	dtype: object -> datetime64
T4	Aggregate payments	Split payments collapsed to 1 row per order	~103k rows -> ~99k rows
T5	Build master table	5 tables merged into one wide DataFrame	~99k rows, 20+ columns
T6	Feature engineering	3 new columns: delivery_days, delay_days, is_late	3 analysis-ready features added

Phase 0 & 1 complete. The master DataFrame is ready for Phase 2 EDA. Run all cells top to bottom in Google Colab. Only update the FOLDER path in Task 1 if your files are stored elsewhere.